

RSA ENCRYPTION CHEAT SHEET

Asymmetric Cryptography, Mathematics & Implementation

SECURITY STUDY GUIDE

Based on MuirlandOracle Reference

1. ASYMMETRIC VS. SYMMETRIC

Concept	Asymmetric (RSA)	Symmetric (AES)
Keys	2 Keys (Public & Private)	1 Key (Shared Secret)
Security	No secret exchange needed	Key exchange risk
Usage	Signatures, SSH, Handshakes	Bulk Data Encryption

Alice & Bob Analogy: Bob distributes unlocked boxes (**Public Key**) nationwide. Alice seals a message inside. Only Bob's physical key (**Private Key**) can unlock it.

2. MATHEMATICAL FOUNDATIONS

- Coprimality:** Two integers a and b are coprime if $\gcd(a, b) = 1$. (e.g., 8 and 15 are coprime).
- Euler's Totient (ϕ):** Counts integers up to n coprime to n . For primes p, q :
$$\phi(n) = (p - 1)(q - 1)$$
- Modular Arithmetic:** Clock arithmetic evaluating remainders: $a \equiv r \pmod{m}$.

3. RSA KEY GENERATION STEPS

- Select two distinct prime numbers p and q .
- Calculate modulus $n = p \times q$.
- Calculate Euler's totient $\phi(n) = (p - 1)(q - 1)$.
- Choose public exponent e such that:
$$1 < e < \phi(n) \text{ and } \gcd(e, \phi(n)) = 1$$
- Calculate private exponent d such that:
$$d \times e \equiv 1 \pmod{\phi(n)}$$

GENERATED KEY PAIRS

Public Key: (e, n) | Private Key: (d, n)

4. WORKED NUMERICAL EXAMPLE

Prime Setup: $p = 11, q = 13$

- $n = 11 \times 13 = 143$
- $\phi(n) = (11 - 1)(13 - 1) = 10 \times 12 = 120$
- Pick $e = 7$ (since $\gcd(7, 120) = 1$)
- Solve $d \times 7 \equiv 1 \pmod{120} \rightarrow$ Choose $d = 223$

ENCRYPTION & DECRYPTION OPERATIONS

Encrypt Message ($M = 24$):

$$C = 24^7 \pmod{143} = 106$$

Decrypt Cipher ($C = 106$):

$$M = 106^{223} \pmod{143} = 24$$

5. EUCLIDEAN ALGORITHM (GCD)

Computes the Greatest Common Divisor efficiently via successive division: $a = b \cdot k + r$

Manual Example (GCD of 24 & 16):

$$\begin{aligned} 24 &= 16(1) + 8 && \rightarrow a=16, b=8 \\ 16 &= 8(2) + 0 && \rightarrow \text{Remainder is } 0, \text{ GCD} = 8 \end{aligned}$$

Recursive Python Implementation:

```
def gcd(a, b):
    if b == 0:
        return a
    return gcd(b, a % b)
```

6. PYTHON KEY GENERATOR

```
import random

def prime_finder():
    test = random.randrange(10, 100)
    for i in range(2, test):
        if test % i == 0:
            return prime_finder()
    return test

# 1. Setup Parameters
p, q = prime_finder(), prime_finder()
n = p * q
phi = (p - 1) * (q - 1)

# 2. Public Key (e)
pub_keys = [i for i in range(2, phi)
             if gcd(i, phi) == 1 and gcd(i, n) == 1]
e = random.choice(pub_keys[:100])

# 3. Private Key (d)
priv_keys = []
i = 2
while len(priv_keys) < 5:
    if (i * e) % phi == 1:
        priv_keys.append(i)
    i += 1
d = random.choice(priv_keys)

print(f"Public Key: ({e}, {n})")
print(f"Private Key: ({d}, {n})")
```

7. C++ IMPLEMENTATION SNIPPET

```
#include <iostream>
#include <vector>

int gcd(int a, int b) {
    return (b == 0) ? a : gcd(b, a % b);
}

// Key loop snippet for private key d
vector<unsigned int> priv_keys;
unsigned int i = phi + 1;
while (priv_keys.size() < 5) {
    if ((i * e) % phi == 1) priv_keys.push_back(i);
    i++;
}
```

8. SECURITY & REAL-WORLD NOTES

- **Factorization Hardness:** RSA security relies on the difficulty of factoring huge primes ($n = p \times q$).
- **Key Sizes:** Real-world RSA requires **2048** to **4096-bit** keys (not small demo integers).
- **Padding Protocols:** Raw RSA is vulnerable; real systems use OAEP padding.